



Project: "Neuro-Search" (Mini-JEPA)

The Pitch: A "Zero-Shot Video Search Engine" that understands the *content* of a video without needing tags or captions. You will build a Joint Embedding Predictive Architecture (JEPA) from scratch that maps video segments and text queries into a shared "meaning space."

Hardware Strategy (Crucial for your laptops):

- **Coding:** Do all coding on your local laptops (VS Code)
- **Debugging:** Run small tests locally (use `batch_size=2` or just 10 images). It will run fine
- **Training:** Do **NOT** train on your laptops. Push your code to GitHub, open **Google Colab (Free Tier)**, pull the code, and use the free T4 GPU there for the heavy lifting.

Project Structure: "The Swarm" Approach

In this model, the team moves together through phases. Everyone writes PyTorch code. Everyone writes API code.

Phase 1: The "Building Blocks" Sprint

Goal: Everyone builds a *piece* of the Neural Network. You cannot finish this phase until all four pieces plug together without crashing.

Learning Objective: Understanding Modular Deep Learning & Tensor Shapes.



Assignments (Parallel PyTorch Work)

- **Nawfal (The Vision Wrapper):**
 - **Task:** Write a Python class `VisionModule` that loads a pre-trained Vision Transformer (like `timm` or `CLIP`).
 - **The constraint:** It must accept a raw image tensor and output a fixed vector (e.g., shape `[Batch, 512]`). It *must* freeze the weights (set `requires_grad=False`) so it runs on your laptops.
 - **Skill Gained:** Transfer Learning, Hugging Face Transformers.
- **Ali (The Text Wrapper):**
 - **Task:** Write a Python class `TextModule` that loads a pre-trained BERT/MiniLM.
 - **The constraint:** It must handle "tokenization" inside the class. Input is a list of strings `["cat", "dog"]`, output is a vector `[Batch, 384]`.
 - **Skill Gained:** NLP (Natural Language Processing), Tokenizers.
- **Abdullah (The "Predictor" Brain):**
 - **Task:** This is the core JEPA logic. Write a `PredictorNetwork` class (a 4-layer Neural Network).

- **The constraint:** It must take the vectors from Nawfal and Ali, concatenate them (or process them), and output a "prediction" vector. You need to decide the math: ReLU? BatchNorm? Dropout?
- **Skill Gained:** Designing Neural Networks from scratch in PyTorch.
- **Ansab (The Loop Master):**
 - **Task:** Write the `TrainStep` function and the `Loss` function.
 - **The constraint:** You need to research "Cosine Similarity Loss". You will write the code that takes the output from Abdullah and compares it to the ground truth to calculate the "Error".
 - **Skill Gained:** Optimization Mathematics, Loss Landscapes.

🚩 The "Merge Day"

You all meet. You create a file `main.py`. You import everyone's classes.

Python

```
# main.py
img_vec = Nawfal.VisionModule(image)
txt_vec = Ali.TextModule(text)
pred = Abdullah.PredictorNetwork(img_vec, txt_vec)
loss = Ansab.LossFunction(pred, target)
```

If this runs without a "Dimension Mismatch Error," you have passed Phase 1.

Phase 2: The "Hyperparameter Tournament"

Goal: Train the model to actually work. Since you have 4 people and 4 Google accounts (4 Free Colab GPUs), you will do **Parallel Experimentation**.

Learning Objective: Experiment tracking, Overfitting/Underfitting, Data Science.

🧪 The Experiments (Distributed Training)

Instead of one person training, **everyone trains a variation** to see what works best. This mimics a real Research Lab environment.

- **Nawfal(The "Data" Variant):**
 - **Experiment:** Train using *Data Augmentation* (rotate images, blur images, change colors) vs. No Augmentation.
 - **Hypothesis:** Does making the images harder to see make the AI smarter?
- **Ali (The "Architecture" Variant):**
 - **Experiment:** Change Abdullah's network. Try a wide network (Layer size 1024) vs. a deep network (8 layers).

- **Hypothesis:** Does a "deeper" brain understand video better than a "wider" brain?
- **Abdullah (The "Optimizer" Variant):**
 - **Experiment:** Compare AdamW optimizer vs. SGD (Stochastic Gradient Descent) with different Learning Rates (0.001 vs 0.0001).
 - **Hypothesis:** How fast can we learn without crashing (diverging)?
- **Ansab (The "Objective" Variant):**
 - **Experiment:** Try "Hard Negative Mining" (teaching the AI specifically on confusing examples, like a wolf vs. a husky) vs. Random Sampling.
 - **Hypothesis:** Does showing hard examples improve accuracy?



The Review (End of Week 4)

- Everyone brings their "Loss Curves" (graphs from Colab).
- You compare results.
- **Winner:** The configuration with the lowest Validation Loss becomes the "Official Team Model" for Phase 3.
- *Everyone puts "Conducted ablation studies on Hyperparameters" on their CV.*

Phase 3: The "Full Stack" Deployment

Goal: Wrap the model in a product. We will split the *features* of the website so everyone touches Backend & Frontend.

Learning Objective: API Development (FastAPI), System Integration.



Assignments (Vertical Feature Slices)

- **Nawfal(The Search Engine):**
 - **Task:** Build the API endpoint `/search`. It takes a text string, converts it to vector (using the Model), and finds the nearest video in the database.
 - **Tech:** FastAPI + Numpy/FAISS.
- **Ali (The Video Ingest):**
 - **Task:** Build the API endpoint `/upload`. It accepts a video file, breaks it into frames, runs the Model, and saves the vectors to the database.
 - **Tech:** OpenCV + FastAPI + File Handling.
- **Abdullah (The Frontend Dashboard):**
 - **Task:** Build the UI using **Streamlit** (Python UI library). It needs a search bar and a video player grid to display results from N's search.
 - **Tech:** Streamlit / Gradio.
- **Ansab (The "Explainability" & Ops):**

- **Task:** Build a "Confidence Score" UI. When a result is shown, display *why* (e.g., "98% match"). Also, set up the **Docker Container** so this whole thing runs on the cloud.
- **Tech:** Docker, Math Visualization.



The CV Payoff (The "Result")

By doing it this way, **all four of you** can put the *entire* stack on your CV, because you all touched the code during the Merge and Review sessions.

Project Title: Neuro-Search (Semantic Video Understanding Engine)

- **Tech Stack:** PyTorch, Transformers, FastAPI, Docker, FAISS.
- **Description:**
 - Co-developed a **Joint Embedding Predictive Architecture** (JEPA) for zero-shot video retrieval.
 - **[For Nawfal]:** Engineered the Vision Encoder pipeline using transfer learning (CLIP) and optimized vector inference speed by 40%.
 - **[For Ali]:** Implemented NLP tokenization pipelines and managed the multi-modal data ingestion architecture.
 - **[For Abdullah]:** Designed the core MLP predictor network and developed the frontend dashboard for real-time semantic search.
 - **[For Ansab]:** Led the hyperparameter optimization strategy (Ablation studies) and implemented the Dockerized deployment pipeline.
 - **Shared Bullet Point:** Achieved a **Top-5 Accuracy of X%** on the validation set through collaborative distributed training experiments.



Hardware Survival Guide (Specifics)

1. **RAM:** You have 16GB. This is good. When coding locally, use `batch_size=1` to prevent crashing.
2. **Storage:** You have 1TB. You are fine to download datasets, but delete the raw `.mp4` files after you process them into tensors to keep things clean.
3. **GPU:** Since you don't have NVIDIA GPUs locally, you must install the **CPU version** of PyTorch locally:

```
pip install torch torchvision --index-url https://download.pytorch.org/whl/cpu
```

(This saves space and prevents errors looking for CUDA drivers that don't exist).